

Communauté Française de Belgique
Institut des Carrières Commerciales
Ville de Bruxelles
Waterside
Quai de Willebroeck, 33
1000 Bruxelles

Travail réalisé pour l'unité d'enseignement de
« **Projet d'Intégration de Développement** »

DOCUMENTATION TECHNIQUE

Projet « Réservations »

Application web de réservation de spectacles - ReserShows

Étudiants	Yassine Boussaid Amine Radi
Année académique	2025 - 2026
Version du document	1.0
Date	24 août 2026

1. Objet, périmètre et état de la version

Cette documentation décrit l'architecture, l'environnement de développement, les fonctionnalités, la sécurité, les API, les échanges CSV/RSS et les procédures d'installation du projet Réservations. Elle est destinée à un lecteur technique devant reprendre, tester ou faire évoluer l'application.

Élément	Valeur
Nom fonctionnel	Réservations / ReserShows
Type	Application web SPA + API REST
Backend	Java 17 - Spring Boot 3.3.5
Frontend	React + Vite (versions déclarées « latest », non figées dans package.json)
Base de données	MySQL - schéma reservations_db
Port backend local	8085
Port frontend local	5173 par défaut avec Vite
Documentation API	Swagger UI : http://localhost:8085/swagger-ui/index.html

État de la version analysée

Le backend est identifié comme 0.0.1-SNAPSHOT. Les archives fournies ne contiennent pas l'historique Git ni un fichier de verrouillage npm. La présente documentation décrit donc l'état exact du code fourni au 24/08/2026, sans inventer de version frontend absente des sources.

1.1 Architecture générale

Couche	Technologie	Responsabilités principales
Frontend	React, React Router, Vite, CSS	Navigation SPA, formulaires, catalogue, profil, espace admin, appels HTTP.
API	Spring Boot / Spring MVC	Endpoints REST, validation, orchestration des services, Swagger/OpenAPI.
Métier	Services Java	Réservations, disponibilité, avis, import, statistiques, affiliation.
Persistence	Spring Data JPA / Hibernate	Entités, repositories, relations et accès MySQL.
Sécurité	Spring Security + JWT + BCrypt	Authentification stateless, Bearer token, rôles et CORS.
Intégrations	Brussels OpenData, CSV, RSS	Import de données externes, échange de catalogue, syndication XML.

1.2 Modèle métier

Le modèle de données couvre le cycle complet d'un spectacle : localisation, spectacle, représentation, réservation, prix, avis, artistes et accès partenaire.

Bloc	Entités principales
Utilisateurs	users, roles
Catalogue	localities, locations, shows, representations, prices
Réservation	reservations, representation_reservations
Artistes	artists, artist_types, artist_type_assignments, collaborations
Avis	reviews
Affiliation	affiliate_plans, api_keys

2. Méthodologie et techniques de développement

2.1 Langages, frameworks et bibliothèques

Composant	Version / état	Usage
Java	17	Langage backend.
Spring Boot	3.3.5	Socle applicatif et configuration.
Spring Web	via Spring Boot 3.3.5	API REST / MVC.
Spring Data JPA	via Spring Boot 3.3.5	Repositories et persistance.
Spring Security	via Spring Boot 3.3.5	Authentification et filtre de sécurité.
JWT	0.12.6	Création et validation des JWT.
springdoc-openapi	2.5.0	Swagger UI et document OpenAPI.
MySQL Connector/J	géré par Spring Boot	Connexion JDBC à MySQL.
Lombok	géré par Spring Boot	Réduction du code répétitif Java.
React / React DOM	« latest » dans package.json	Interface utilisateur.
React Router DOM	« latest »	Routage frontend.
Vite	« latest »	Serveur de développement et build frontend.
lucide-react	« latest »	Icônes d'interface.

2.2 Techniques de développement

- Architecture en couches : controller → service → repository → base de données.
- DTO d'entrée/sortie pour découpler l'API des entités JPA.
- Validation Jakarta Bean Validation (@NotBlank, @NotNull, @Min, @Max, @Email, etc.).
- API REST avec JSON, codes HTTP et traitement centralisé des erreurs.
- Authentification stateless par JWT transmis dans l'en-tête Authorization: Bearer <token>.
- Hachage des mots de passe avec BCrypt.
- Recherche, tri et pagination du catalogue des spectacles.
- Transactions sur les opérations de réservation et d'annulation afin de maintenir la capacité des représentations.
- Import/export CSV et intégration d'une API Open Data.
- Documentation interactive OpenAPI / Swagger.

2.3 Logiciels et environnement

Outil / logiciel	Rôle
IntelliJ IDEA	Métadonnées .idea présentes dans le backend ; édition, exécution et debug Java.
Maven Wrapper 3.9.15	Compilation, tests et exécution du backend sans installation Maven globale.
Node.js + npm	Installation des dépendances et exécution Vite.
MySQL Server	SGBD local pour reservations_db.
Navigateur moderne	Utilisation du frontend, DevTools et tests fonctionnels.
Swagger UI / Postman	Tests manuels des endpoints REST.
Git + GitHub	Versionnement et publication obligatoires dans le cadre du cours.

2.4 Gestion de projet

Le projet est structuré de manière incrémentale par fonctionnalités : authentification, catalogue, réservations, avis, administration, import/export et intégrations. Git/GitHub constitue le support de versionnement attendu. L'archive remise ne contient pas le dossier .git, donc la régularité des commits doit être contrôlée directement sur le dépôt GitHub avant la remise.

Point de conformité

La consigne de seconde session impose un versionnement Git régulier et une publication sur GitHub. Le document inclut une zone dédiée à l'URL du dépôt en section 11.

3. Fonctionnalités implémentées et URLs

3.1 Routes frontend

URL frontend	Accès	Fonction
/	Public	Accueil et accès au catalogue.
/shows	Public	Catalogue avec recherche, filtre bookable, tri et pagination.
/shows/:id	Public / membre	Détail du spectacle, représentations, disponibilité et avis ; réservation/avis si connecté.
/login	Public	Connexion.
/signup	Public	Création d'un compte MEMBER.
/profile	Authentifié	Consultation et modification du profil.
/reservations/me	Authentifié	Liste des réservations du compte courant.
/admin	ADMIN côté frontend	Tableau de bord d'administration.
/admin/:resource	ADMIN côté frontend	CRUD générique sur les ressources administratives.

3.2 Fonctionnalités métier

#	Fonctionnalité	État technique	Endpoint principal
1	Inscription d'un utilisateur	Implémentée	POST /api/auth/signup
2	Connexion JWT	Implémentée	POST /api/auth/login
3	Profil utilisateur	Implémentée	GET /api/auth/me ; PUT /api/users/{id}
4	Catalogue spectacles : recherche/tri/pagination	Implémentée	GET /api/shows
5	Gestion spectacles et données de référence	Implémentée côté API	CRUD /api/shows, /locations, /artists, etc.
6	Gestion représentations et disponibilité	Implémentée côté API	/api/representations ; /{id}/availability
7	Création et annulation de	Implémentée côté backend	POST /api/reservations ;

#	Fonctionnalité	État technique	Endpoint principal
	réservations		PATCH /{id}/cancel
8	Avis et modération	Implémentée côté backend	/api/reviews ; publish/unpublish
9	Import / export CSV	Implémentée côté backend	/api/admin/csv/shows/import export
10	Import Brussels OpenData	Implémentée côté backend	POST /api/admin/external-shows/import
11	Flux RSS des représentations futures	Implémentée	GET /api/rss/upcoming-representations
12	API partenaire par clé	Implémentée	GET /api/affiliate/shows
13	Gestion clés API et plans d'affiliation	Implémentée côté backend	/api/api-keys ; /api/affiliate-plans
14	Statistiques générales et ventes	Implémentée côté backend	/api/statistics ; /shows/{id}/sales

Important - intégration frontend/backend

Plusieurs fonctions existent bien côté backend mais leur écran frontend n'est pas encore totalement aligné avec les DTO ou paramètres attendus. Ces écarts sont détaillés en section 10 afin de ne pas présenter comme « fonctionnel de bout en bout » ce qui nécessite encore une correction.

4. Sécurité : authentification, autorisations et profils

4.1 Authentification

L'application utilise Spring Security en mode stateless. Après inscription ou connexion, le backend retourne un JWT valable 24 heures. Le frontend conserve le token dans localStorage et l'envoie dans l'en-tête Authorization pour les requêtes protégées.

```
Authorization: Bearer <JWT>
Content-Type: application/json
```

Mécanisme	Implémentation
Mots de passe	Hachage BCrypt via BCryptPasswordEncoder.
JWT	JJWT 0.12.6 ; subject = username ; expiration = 86 400 000 ms (24 h).
Session	Aucune session serveur ; SessionCreationPolicy.STATELESS.
CSRF	Désactivé, cohérent avec l'API stateless mais à revalider selon le mode de stockage du token.
CORS	Origines locales autorisées : http://localhost:5173 et http://localhost:3000.
Validation	DTO validés avec Jakarta Validation.

4.2 Rôles prévus

Rôle	Créé automatiquement	Usage prévu
ADMIN	Oui (rôle seulement)	Administration du catalogue et des comptes.
MEMBER	Oui (rôle seulement)	Rôle attribué automatiquement à l'inscription.
PRODUCER	Oui (rôle seulement)	Producteur / gestion métier future.
CRITIC	Oui (rôle seulement)	Modération / critique future.
AFFILIATE	Oui (rôle seulement)	Accès partenaire / affiliation.

4.3 Autorisations réellement appliquées dans la version fournie

Zone	Protection backend actuelle
POST /api/auth/signup, /login	Public.
GET /api/auth/me	JWT obligatoire.
/api/reservations/**	JWT obligatoire.
/api/reviews/**	JWT obligatoire.
/api/affiliate/shows	Public au sens Spring Security, mais contrôle X-API-KEY dans le service.
Swagger, RSS, catalogue et référentiels	Public.
Endpoints /api/admin/**	Actuellement permitAll côté backend.
CRUD /api/users, /roles, /shows, etc.	Actuellement permitAll côté backend.

Écart de sécurité à corriger

Le frontend masque l'administration aux non-ADMIN avec RequireAdmin, mais le backend n'applique pas encore de règle hasRole('ADMIN') sur les routes d'administration. Une interface cachée n'est pas une autorisation de sécurité. Avant une mise en production, les endpoints sensibles doivent être protégés côté Spring Security.

4.4 Procédure d'identification selon les profils

L'archive ne fournit aucun utilisateur de démonstration préchargé. DataSeeder crée uniquement les cinq rôles. Il est donc impossible d'indiquer un login/mot de passe réel sans inventer des identifiants.

Profil	Procédure actuelle	Identifiants à remettre au correcteur
MEMBER	Créer un compte via /signup ; le rôle MEMBER est attribué automatiquement.	À définir dans la base de démonstration.
ADMIN	Créer/modifier un utilisateur et lui associer le rôle ADMIN.	À compléter avant remise.
PRODUCER / CRITIC / AFFILIATE	Créer un utilisateur puis associer le rôle correspondant.	À compléter uniquement si ces profils sont utilisés en démonstration.

Recommandation pour la remise

Créer au minimum deux comptes de démonstration stables (MEMBER et ADMIN), avec des mots de passe dédiés au projet, puis reporter ces identifiants dans cette section.

4.5 Secrets et configuration

- Le secret JWT est actuellement présent en clair dans application.properties : il doit être externalisé dans une variable d'environnement ou un secret de déploiement.
- La configuration locale MySQL utilise root/root : acceptable uniquement pour un environnement de démonstration isolé ; à remplacer en production.
- Les clés API partenaires sont générées avec SecureRandom et encodées en Base64 URL-safe.

5. API consommée : Brussels OpenData

Le backend consomme l'API Open Data de Bruxelles pour importer des lieux / données culturelles dans le catalogue de spectacles. L'URL est configurable par la propriété external.shows.api-url.

```
https://opendata.brussels.be/api/explore/v2.1/catalog/datasets/  
lieux_culturels_touristiques_evenementiels_visitbrussels_vbx/records?limit=20
```

5.1 Déclenchement

```
POST /api/admin/external-shows/import?defaultLocationId=<id>
```

Le paramètre defaultLocationId doit référencer un lieu existant. Chaque élément importé est associé à ce lieu par défaut.

5.2 Exploitation des données

Donnée interne	Champs externes recherchés
title	title, name, nom, naam, designation, denomination, label, event_name, translations_*_name
description	description, summary, body, address/adresse, translations_*_address_line1, municipality, category
posterUrl	posterUrl, image, image_url, poster, photo, thumbnail, media
slug	Généré à partir du titre + id externe si disponible
bookable	true par défaut
price	0.00 par défaut
location	defaultLocationId fourni à l'appel

Le service accepte plusieurs formes de réponse JSON : tableau direct ou objets contenant results, records ou data. Les éléments sans titre sont ignorés.

6. API produite : documentation des endpoints

Base locale de l'API : http://localhost:8085. Les réponses métier sont majoritairement en JSON. Les endpoints peuvent être consultés et testés via Swagger UI.

6.1 Authentification et catalogue

Méthode	Endpoint	Auth.	Corps / paramètres	Rôle
POST	/api/auth/signup	Public	username, email, password, confirmPassword, firstname, lastname, language	Créer un MEMBER + JWT
POST	/api/auth/login	Public	usernameOrEmail, password	Connexion + JWT

Méthode	Endpoint	Auth.	Corps / paramètres	Rôle
GET	/api/auth/me	JWT	-	Profil courant
GET	/api/shows	Public	search, locationId, bookable, page, size, sortBy, direction	Catalogue filtré/paginé
GET	/api/shows/{id}	Public	id	Détail spectacle
POST	/api/shows	Public actuellement	locationId, title, posterUrl, bookable, price, description	Créer spectacle
PUT	/api/shows/{id}	Public actuellement	mêmes champs	Modifier spectacle
DELETE	/api/shows/{id}	Public actuellement	id	Supprimer spectacle

6.2 Réservations, représentations et avis

Méthode	Endpoint	Auth.	Entrée principale	Fonction
GET	/api/representations	Public	-	Lister les représentations
GET	/api/representations/{id}	Public	id	Détail représentation
POST	/api/representations	Public actuellement	showId, locationId, date, time, capacity, bookedSeats	Créer
PUT	/api/representations/{id}	Public actuellement	mêmes champs	Modifier
DELETE	/api/representations/{id}	Public actuellement	id	Supprimer
GET	/api/representations/{id}/availability	Public	id	Capacité / places disponibles
GET	/api/reservations	JWT	-	Lister toutes les réservations
GET	/api/reservations/me	JWT	-	Réservations de l'utilisateur courant
GET	/api/reservations/{id}	JWT	id	Détail réservation
POST	/api/reservations	JWT	representationId, priceId, quantity	Créer réservation
PATCH	/api/reservations/{id}/cancel	JWT	id	Annuler sa réservation
GET	/api/reviews	JWT actuellement	-	Lister les avis
GET	/api/reviews/{id}	JWT	id	Détail avis
POST	/api/reviews	JWT	showId, rating (1-5), comment	Créer avis non publié
PATCH	/api/reviews/{id}/publish	JWT	id	Publier avis
PATCH	/api/reviews/{id}/unpublish	JWT	id	Dépublier avis
DELETE	/api/reviews/{id}	JWT	id	Supprimer avis

6.3 CRUD de référence et administration

Ressource	Base URL	Opérations
Utilisateurs	/api/users	GET, GET/{id}, POST, PUT/{id}, DELETE/{id}
Rôles	/api/roles	GET, GET/{id}, POST, PUT/{id}, DELETE/{id}
Localités	/api/localities	GET, GET/{id}, POST, PUT/{id}, DELETE/{id}
Lieux	/api/locations	GET, GET/{id}, POST, PUT/{id}, DELETE/{id}
Artistes	/api/artists	GET, GET/{id}, POST, PUT/{id}, DELETE/{id}
Types d'artistes	/api/artist-types	GET, GET/{id}, POST, PUT/{id}, DELETE/{id}
Affectations artiste/type	/api/artist-type-assignments	GET, GET/{id}, POST, PUT/{id}, DELETE/{id}
Collaborations	/api/collaborations	GET, GET/{id}, POST, PUT/{id}, DELETE/{id}
Prix	/api/prices	GET, GET/{id}, POST, PUT/{id}, DELETE/{id}
Plans d'affiliation	/api/affiliate-plans	GET, GET/{id}, POST, PUT/{id}, DELETE/{id}
Clés API	/api/api-keys	GET, GET/{id}, POST, PATCH enable/disable, DELETE/{id}

6.4 Intégrations, statistiques et API partenaire

Méthode	Endpoint	En-tête / paramètre	Fonction
POST	/api/admin/csv/shows/import	multipart file	Importer des spectacles CSV
GET	/api/admin/csv/shows/export	-	Télécharger shows.csv
POST	/api/admin/external-shows/import	defaultLocationId	Importer Brussels OpenData
GET	/api/rss/upcoming-representations	-	Flux RSS 2.0 XML
GET	/api/statistics	-	Statistiques globales
GET	/api/statistics/shows/{showId}/sales	showId	Ventes par spectacle
GET	/api/affiliate/shows	X-API-KEY	Catalogue partenaire selon plan

Documentation interactive

Swagger UI : <http://localhost:8085/swagger-ui/index.html>

OpenAPI JSON : <http://localhost:8085/v3/api-docs>

7. Flux RSS produit

Le flux RSS 2.0 expose les représentations dont la date n'est pas antérieure à la date courante. Chaque item contient le titre du spectacle, le lieu, la date/heure et un identifiant `representation-<id>`.

```
GET http://localhost:8085/api/rss/upcoming-representations
```

Point à corriger

Le champ <link> généré par RssFeedService contient actuellement http://localhost:8080 alors que le backend écoute sur 8085. Cette valeur doit être rendue configurable ou corrigée.

8. Import / export CSV

8.1 Import

Endpoint : POST /api/admin/csv/shows/import, avec un fichier multipart nommé file.

```
title,posterUrl,bookable,price,description,locationId
Ayiti,/wrapped/imgs/ayiti.jpg,true,25.00,A theatre show,1
```

Le service ignore la première ligne (en-tête), exige au moins six colonnes et vérifie que locationId existe. L'encodage lu est UTF-8.

8.2 Export

Endpoint : GET /api/admin/csv/shows/export. Le serveur renvoie un téléchargement shows.csv avec les colonnes : id, title, posterUrl, bookable, price, description, locationId, locationDesignation.

Limite technique du parseur d'import

L'import utilise `line.split(",", -1)` et ne gère donc pas correctement les virgules échappées/quotées dans les champs. L'export, lui, ajoute des guillemets si nécessaire. Pour une version robuste, utiliser Apache Commons CSV ou OpenCSV.

9. Procédure d'installation et de déploiement local

9.1 Prérequis

- JDK 17.
- MySQL Server accessible localement.
- Node.js + npm.
- Git (si installation depuis le dépôt).
- Ports libres : 8085 pour le backend et 5173 pour Vite.

9.2 Base de données

1. Créer la base MySQL `reservations_db`.

```
CREATE DATABASE reservations_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

2. Vérifier la configuration dans `src/main/resources/application.properties`.

```
spring.datasource.url=jdbc:mysql://localhost:3306/reservations_db
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
server.port=8085
```

Hibernate crée/met à jour les tables au démarrage. Le fichier `src/main/resources/static/db.sql` fournit également un schéma SQL de référence.

9.3 Démarrage du backend

```
cd backend
# Windows
.\mvnw.cmd clean test
.\mvnw.cmd spring-boot:run

# Linux / macOS
./mvnw clean test
./mvnw spring-boot:run
```

Le backend doit répondre sur `http://localhost:8085`.

9.4 Démarrage du frontend

```
cd reservations-react-frontend
npm install
copy .env.example .env          # Windows
# cp .env.example .env          # Linux/macOS
npm run dev
VITE_API_BASE_URL=http://localhost:8085
```

Ouvrir ensuite l'URL indiquée par Vite, généralement <http://localhost:5173>.

9.5 Vérification après installation

- Ouvrir </swagger-ui/index.html> et vérifier que les endpoints sont chargés.
- Créer un compte via </signup>, puis vérifier [GET /api/auth/me](/api/auth/me).
- Créer au moins un lieu, un spectacle, un prix et une représentation.
- Tester la disponibilité d'une représentation.
- Tester un endpoint CSV, RSS et l'import OpenData.
- Tester le frontend avec le backend actif et contrôler la console réseau du navigateur.

9.6 Déploiement production

Aucun Dockerfile, docker-compose, pipeline CI/CD ou configuration d'hébergement public n'est présent dans les archives. Le déploiement documenté et reproductible est donc local. Pour un déploiement serveur, prévoir au minimum : build Vite, reverse proxy HTTPS, JAR Spring Boot, base MySQL dédiée, variables d'environnement pour les secrets, CORS adapté au domaine public et service de supervision.

10. Développements futurs et points de stabilisation

Les éléments ci-dessous sont issus d'une lecture croisée du frontend et du backend fournis. Ils doivent être traités en priorité pour obtenir un parcours complet cohérent.

Priorité	Point	Constat actuel	Action recommandée
P0	Réservation depuis la fiche spectacle	Le frontend envoie <code>representationId</code> + <code>seats</code> , alors que le backend exige <code>representationId</code> + <code>priceId</code> + <code>quantity</code> .	Ajouter le choix du tarif et envoyer <code>priceId/quantity</code> .
P0	Autorisation admin backend	Les routes admin et plusieurs CRUD sont <code>permitAll</code> ; seul le frontend masque l'écran.	Appliquer <code>hasRole('ADMIN')</code> / règles serveur.
P0	Import OpenData depuis l'admin	Le bouton frontend n'envoie pas <code>defaultLocationId</code> , paramètre obligatoire.	Demander/choisir un lieu et ajouter ? <code>defaultLocationId=</code> .
P1	Filtrage représentations par spectacle	Le frontend envoie <code>showId</code> mais le controller retourne toutes les représentations.	Ajouter <code>@RequestParam showId</code> et requête <code>repository</code> .
P1	Filtrage avis par spectacle	Le frontend envoie <code>showId</code> mais le controller retourne tous les avis.	Ajouter filtre <code>showId</code> ; exposer publiquement uniquement les avis publiés.
P1	Plans d'affiliation dans l'admin	Champs frontend <code>name/description/monthlyQuota/price</code> ≠ DTO backend <code>planName/apiLimit/monthlyPrice</code> .	Aligner les noms et formulaires.
P1	Comptes de démonstration	Aucun utilisateur n'est seedé.	Créer comptes MEMBER/ADMIN reproductibles pour la correction.
P1	Secrets	JWT secret et mot de passe DB sont dans <code>application.properties</code> .	Variables d'environnement + profils Spring.
P2	CSV robuste	Import par <code>split(',')</code> simple.	Utiliser une bibliothèque CSV.
P2	RSS	Lien interne du feed pointe vers <code>localhost:8080</code> .	Rendre base URL configurable.
P2	Versions frontend	Dépendances déclarées « latest ».	Ajouter <code>package-lock.json</code> et figer les versions.
P2	Tests	Tests backend limités ; aucun test frontend fourni.	Ajouter tests services/controllers et tests React/E2E.
P3	Évolutions produit	Païement, email, temps réel, cache, rate limiting, Docker/cloud.	Planifier après stabilisation du socle.

11. URLs de remise et accès de démonstration

Élément demandé	Valeur
URL de développement - dépôt GitHub	À COMPLÉTER : l'URL n'est pas contenue dans les archives fournies.
URL de déploiement - site en ligne	À COMPLÉTER, ou indiquer explicitement « non déployé publiquement » si le cours l'accepte.
Backend local	http://localhost:8085
Frontend local	http://localhost:5173
Swagger UI	http://localhost:8085/swagger-ui/index.html
OpenAPI JSON	http://localhost:8085/v3/api-docs
Flux RSS	http://localhost:8085/api/rss/upcoming-representations

À compléter avant dépôt

La consigne exige explicitement l'URL GitHub et l'URL de déploiement. Ces deux informations ne peuvent pas être déduites des archives ZIP. Elles doivent être ajoutées ici avant la remise finale.

11.1 Comptes de démonstration

Profil	Login / e-mail	Mot de passe	Statut
MEMBER	À COMPLÉTER	À COMPLÉTER	Aucun compte fourni dans l'archive
ADMIN	À COMPLÉTER	À COMPLÉTER	Aucun compte fourni dans l'archive

12. Guide de reprise rapide pour un développeur

1. Installer JDK 17, MySQL, Node.js/npm et Git.
2. Créer reservations_db et vérifier application.properties.
3. Lancer le backend avec le Maven Wrapper sur le port 8085.
4. Créer .env côté frontend avec VITE_API_BASE_URL=http://localhost:8085.
5. Installer les dépendances npm et lancer npm run dev.
6. Créer un utilisateur MEMBER, puis un compte ADMIN de démonstration.
7. Vérifier Swagger, catalogue, disponibilité, RSS, CSV et OpenData.
8. Corriger les écarts P0 de la section 10 avant de déclarer le parcours complet comme validé.
9. Vérifier l'historique GitHub et renseigner les URLs de la section 11.

12.1 Arborescence utile

```
backend/  
  src/main/java/com/pid/backend/  
    config/      # Security, OpenAPI, seeding  
    controller/  # Endpoints REST  
    dto/         # Contrats d'entrée/sortie  
    entity/      # Entités JPA  
    repository/  # Accès aux données  
    security/    # JWT / UserDetails  
    service/     # Logique métier  
  src/main/resources/  
    application.properties  
    static/db.sql  
  
reservations-react-frontend/  
  src/main.jsx   # Routage + pages + appels API  
  src/styles.css  
  .env.example  
  package.json
```

12.2 Conclusion technique

Le projet fournit un socle backend riche et modulaire : authentification JWT, catalogue, réservations, avis, gestion de données, statistiques, Open Data, CSV, RSS et API partenaire. L'architecture Spring Boot/JPA/MySQL est claire et facilement extensible. La priorité avant une remise définitive est de sécuriser les routes administratives, aligner les quelques contrats frontend/backend identifiés, figer les dépendances frontend et fournir des comptes/URLs de démonstration reproductibles.